Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Property Testing using Python's Hypothesis

Pedro Vasconcelos

pbvascon@fc.up.pt

Departamento de Ciência de Computadores, Faculdade de Ciências da Universidade do Porto

April 2026

U.PORTO
FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

EDIH European Digital Innovation Hubs Network

PRR Plano de Recuperação e Resiliência

REPÚBLICA PORTUGUESA

Financiado pela União Europeia NextGenerationEU

CONNECT5

Table of contents

1. Introduction
2. Testing simple functions
3. Practice session 1
4. Testing functions with structured data
5. Practice session 2
6. Testing stateful code
7. Practice session 3
8. Conclusion

Table of contents

1 Introduction

2 Testing simple functions

3 Practice session 1

4 Testing functions with structured data

5 Practice session 2

6 Testing stateful code

7 Practice session 3

8 Conclusion

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Overview

- A long-standing challenge for software engineering is ensuring software correctness
- Formal verification is (still) expensive and rarely used
- *Tests* are the most commonly used practical technique
- *Unit tests* are the industry-standard for checking software components

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

This workshop

- Introduction to *property-based testing*
- An automatic testing methodology that complements unit tests
- We will use Python (but it is available for many other programming languages)
- Many successful applications in both open-source and some industrial projects
- But still not commonly known and under-utilized in practice

Slides and demo code: `https://github.com/pbv/pbt-workshop`

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Intended audience

Familiarity with basic Python programming:

- built-in data types and structures (lists, tuples, dictionaries)
- defining functions and methods
- basics of functional programming (map, filter, lambdas)

Familiarity with basics of testing:

- assertions
- defining unit tests for simple functions
- running tests automatically (pytest, VSCode)

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Unit tests

- Code fragments for testing functions, methods, classes, libraries, etc.
- Express the expected behaviour for specific combinations of inputs

Unit tests (cont.)

Example: testing an integer square root function in Python

```
import math

def isqrt(n):
    return int(math.sqrt(n))

def test_isqrt():
    assert isqrt(0) == 0
    assert isqrt(2) == 1
    assert isqrt(4) == 2
    assert isqrt(5) == 2
    assert isqrt(9) == 3
```

Problems with unit tests

Cognitive bias:

- how can we include an edge case in the tests that we didn't consider in the code?

Poor scaling:

- a few unit tests per feature
- for n features, $O(n)$ unit tests
- but testing *interactions* between features requires $O(n^2), O(n^3), \ldots$ unit tests

Problems with unit tests

Cognitive bias:

- how can we include an edge case in the tests that we didn't consider in the code?

Poor scaling:

- a few unit tests per feature
- for n features, $O(n)$ unit tests
- but testing *interactions* between features requires $O(n^2), O(n^3), \ldots$ unit tests

Solution:

> *"Don't write tests — generate them!"*

John Hughes, co-author of the *QuickCheck* library

Property-based testing

- Write properties instead of specific tests
  - functions that take some arguments and assert something
  - should be universal, i.e. hold for *all* values
  - should define the expected behaviour for *all* cases
- Specify *generators* for the inputs
- The testing framework executes the property with a large number of inputs
  - testing fails if a counter-example is found
  - if no counter-example is found, testing succeeds

Property-based testing (cont.)

- QuickCheck was the first PBT library (for Haskell)
- Hence, PBT is sometimes called "QuickCheck testing"
- Other PBT libraries:

  PropEr for Erlang
  ScalaCheck for Scala
  FsCheck for F#
  JUnit-QuickCheck for Java
  RapidCheck for C++
  Hypothesis for Python

Many others: `https://en.wikipedia.org/wiki/QuickCheck`

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

A lightweight formal method

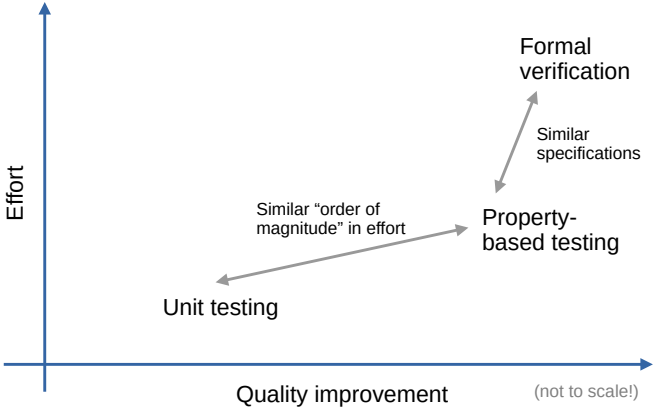
Formal method

A mathematically rigorous technique for validating the actual behaviour of a program against a description of desired behaviors.

Lightweight formal method

One that can be applied successfully by someone who doesn't fully understand it.

PBT vs unit tests vs formal verification



Examples of industrial adoption

- Ericsson used PBT to test telecoms software and found hidden bugs in production code
- Volvo used PBT to test automotive software find bugs in both code and specifications
- Dropbox used PBT to test distributed file synchronization
- Many Python open-source projects are tested using Hypothesis e.g. pytorch, jax, numpy, pandas, mercurial

Table of contents

1 Introduction

2 Testing simple functions

3 Practice session 1

4 Testing functions with structured data

5 Practice session 2

6 Testing stateful code

7 Practice session 3

8 Conclusion

From unit tests to properties

Let us look back to our unit tests for the integer square root:

```
def test_isqrt():
    assert isqrt(0) == 0
    assert isqrt(2) == 1
    assert isqrt(4) == 2
    assert isqrt(5) == 2
    assert isqrt(9) == 3
```

How can we change this into a property?

A false start

```
def test_isqrt(n):  # for some unknown n
    assert isqrt(n) == ????
```

- How can we fill in the blanks?

A false start

```
def test_isqrt(n):  # for some unknown n
    assert isqrt(n) == guess_isqrt(n)
```

- How can we fill in the blanks?
- We need to "guess" the result of `isqrt`

A false start

```
def test_isqrt(n):  # for some unknown n
    assert isqrt(n) == guess_isqrt(n)
```

- How can we fill in the blanks?
- We need to "guess" the result of `isqrt`
- But that is just as much work as the function we are trying to test!

A property about square roots

Let n be some non-negative number and $r = \texttt{isqrt}(n)$; then r should be largest non-negative integer such that $r^2 \leq n$.

Mathematically:

$$r \geq 0 \wedge r^2 \leq n \wedge (r+1)^2 > n$$

A property about square roots (cont.)

Using Python and Hypothesis:

```
from hypothesis import given
import hypothesis.strategies as st

@given(st.integers(min_value=0))
def test_isqrt(n):
    r = isqrt(n)
    assert r>=0 and r**2<=n and (r+1)**2>n
```

(Cue demo.)

A property about square roots (cont.)

- The property failed for a very large number
- This happens because `math.sqrt` converts integers to floating point numbers, thereby losing precision
- We hadn't anticipated this edge case in our unit tests
- How to fix this?

  change the property restricting generation to smaller numbers

```
@given(st.integers(min_value=0,
                   max_value=10_000_000))
```

  change the implementation using an exact algorithm `https://en.wikipedia.org/wiki/Integer_square_root`

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Properties in Hypothesis

- Properties are functions that fail if some expected condition is not met
- Arguments are universally quantified
- For each property:
  - will be test with a large number of random values (100 by default)
  - generated using strategies (defined by the `@given` decorator)
- Module `hypothesis.strategies` provides:
  - predefined strategies for basic types
  - methods for modifying and combining strategies

Strategies for basic values

floats()	generate floating-point numbers
integers()	generate integers
booleans()	generate logical values
text()	generate Unicode strings

You can:

- customize some strategies using parameters (e.g. `min_value` and `max_value` for numbers)
- inspect examples of values drawn from a strategy using the `example` method

Strategies for basic values (cont.)

```
>>> from hypothesis.strategies import integers, booleans
>>> integers(min_value=0, max_value=10).example()
4
>>> integers(min_value=0, max_value=10).example()
7
>>> booleans().example()
True
>>> booleans().example()
False
```

Mapping and filtering strategies

We can use the `map` and `filter` methods to adapt strategies:

- **map** applies a function to every value generated
- **filter** discards values that do not meet a specified condition

Mapping and filtering strategies (cont.)

Examples:

```
# generate only even numbers
integers().map(lambda x:2*x)

# generate only odd numbers
integers().map(lambda x:2*x+1)

# generate only even numbers, alternative
integers().filter(lambda x:x%2==0)
```

Mapping and filtering strategies (cont.)

Caveat:

- `filter` may need to discard many generated values
- if the condition is rarely met, filtering will be slow
- Hypothesis will give up if filtering takes too long

```
>>> integers(max_value=10).filter(lambda x: x>10).example()
hypothesis.errors.Unsatisfiable: ...
```

Strategies are first-class values

- We can give names to strategies and use them later
- This allows defining strategies compositionally and improves test readability

Strategies are first-class values (cont.)

```
from hypothesis import given
import hypothesis.strategies as st

# a strategy for odd integer numbers
odds = st.integers().map(lambda x:2*x+1)

# a simple property:
# adding two odd numbers gives an even number
@given(odds, odds)
def test_add_odd_numbers(x, y):
    assert (x+y)%2 == 0  # check that the sum is even
```

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Integrating Hypothesis with Python tools

- Python `pytest` tool can pick up Hypothesis tests as well as ordinary unit tests
- We can use VSCode `pytest` integration to automatically run property tests
- Tips:
  - Open the source directory with Python files
  - Define tests in files with `test_` prefix
  - Define properties with `test_` prefix

(Cue demo.)

Table of contents

1 Introduction

2 Testing simple functions

3 Practice session 1

4 Testing functions with structured data

5 Practice session 2

6 Testing stateful code

7 Practice session 3

8 Conclusion

Practice session 1

1 Test the integer square test property and fix the failure by limiting the integers generated

2 Introduce an error into the implementation that gives the wrong answer for some arguments e.g.:

```
def isqrt(n):
    if n>1000 and n<2000:
        return n//2   # WRONG
    else:
        return int(math.sqrt(n))
```

Re-run the Hypothesis tests and see what counter-example you get.

Practice session 1 (cont.)

Define and test the following properties using Hypothesis.

3 Adding two even integer numbers always gives an integer result.

4 Adding an even integer number and an odd integer number gives an odd integer number.

5 Integer addition is *commutative*: $x + y$ should always give the same result as $y + x$.

6 Integer addition is *associative*: $(x + y) + z$ should always give the same result as $x + (y + z)$.

7 Floating point addition is associative (spoiler: it is not and Hypothesis will produce a counter-example).

Table of contents

1 Introduction

2 Testing simple functions

3 Practice session 1

4 Testing functions with structured data

5 Practice session 2

6 Testing stateful code

7 Practice session 3

8 Conclusion

Strategies for compound data

lists(s) lists of elements given by strategy s

tuples($s1, s2, \ldots$) lists of elements given by strategies $s1, s2, \ldots$

dictionaries($s1$, $s2$) dictionaries with keys given by strategies $s1$ and values given by $s2$

These strategies take optional parameters to control generation (see the documentation).

Strategies for compound data (cont.)

Some examples:

```
# lists of integers from 0 to 100
lists(integers(min_value=0,max_value=100))

# lists of integers with at least 3 elements
lists(integers(), min_size=3)

# tuples of a boolean and an integer
tuples(booleans(),integers())

# dictionaries from 5 character strings to integers
dictionaries(texts(min_size=4, max_size=4), integers())
```

A simple property on lists

Reversing a list twice should give back the original list.

```
from hypothesis import given
import hypothesis.strategies as st

@given(st.lists(st.integers()))
def test_reverse_twice(x):
    y1 = list(reversed(x))   # first reverse
    y2 = list(reversed(y1)) # second reverse
    assert y2 == x
```

Shrinking

- Hypothesis attempts to simplify the counter-examples found before presenting
- This is useful to remove "noise" from randomly generated data
- Each strategy employs a specific criteria for simplification
- For example, `lists()` attempts to:
  - reduce the list length
  - if that is not possible, shrink some values inside the list
- Hypothesis keeps shrinking while the example still causes the property to fail

An example

A wrong property on lists:

```
@given(st.lists(st.integers()))
def test_reverse_wrong(x):
    assert list(reversed(x)) == x
```

An example

A wrong property on lists:

```
@given(st.lists(st.integers()))
def test_reverse_wrong(x):
    assert list(reversed(x)) == x
```

```
pytest basic.py -k test_reverse_wrong
```

```
E       Falsifying example: test_reverse_wrong(
E           x=[0, 1],
E       )
```

An example

A wrong property on lists:

```
@given(st.lists(st.integers())
def test_reverse_wrong(x):
    assert list(reversed(x)) == x
```

```
pytest basic.py -k test_reverse_wrong
```

```
E        Falsifying example: test_reverse_wrong(
E            x=[0, 1],
E        )
```

- The "smaller" lists `[]`, `[0]`, `[1]` and `[0,0]` are *not* counter-examples
- The counter-example reported is *minimal*
- In general, shrinking will find local minima, not the global minimum

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Composing strategies

- We can use compound strategies to build new strategies for domain-specific applications
- For simplicity, we will only look at the basics in this session
- Hypothesis provides more advanced combinators for building strategies (check my book for more advanced uses)

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Composing strategies (cont.)

Example: a strategy for generating dates.

- years from 1970 to 2100
- months from 1 to 12
- days from 1 to 31

Composing strategies (cont.)

First attempt:

```
years = integers(min_value=1970, max_value=2100)
months = integers(min_value=1, max_value=12)
days = integers(min_value=1, max_value=31)
dates = tuples(years, months, days)
```

Problem: will generate invalid dates e.g. 30th of February.

Composing strategies (cont.)

Solution: filter invalid dates.

```
years = integers(min_value=1970, max_value=2100)
months = integers(min_value=1, max_value=12)
days = integers(min_value=1, max_value=31)
dates = tuples(years, months, days).filter(valid_date)

def valid_date(date):
    (d,m,y) = date
    # check if the date is valid
    # and return True or False
    ...
```

Note: in real code you should use the `dates` strategy from the Hypothesis library.

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

A larger example

Encode and decode to *comma-separated values* (CSV) a table with employee data:

- a numerical identifier
- the employee's name
- the starting work date
- a text field for notes

A larger example (cont.)

Example:

ID	Name	Start Date	Notes
1	Alice Johnson	2022-01-15	Enjoys Python
2	Bob Lee	2021-11-02	Fluent in French
3	Clara Smith	2020-05-30	Handles onboarding, "HR"
4	Dan O'Neil	2023-03-18	Loves branding, PR

A larger example (cont.)

- We will represent the table in Python as a list of tuples
- The `encode` and `decode` functions convert between the Python representation and a CSV string
- Implemented using Python's `csv` library (check the code repository)

```
>>>> encode([(1,'Alice',date(2022,1,15),'Enjoys Python')])
'1,Alice,2022-01-15,Enjoys Python\r\n'
>>>> decode('1,Alice,2022-01-15,Enjoys Python\r\n')
[(1, 'Alice', datetime.date(2022, 1, 15), 'Enjoys Python')]
```

A larger example (cont.)

A round-trip property: encoding followed by decoding should preserve information.

```
ids = st.integers(min_value=1, max_value=100)
names = st.text(min_size=1)  # avoid empty names
starts = st.dates(min_value=date(2000,1,1),
                  max_value=date(2025,1,1))
notes = st.text()
rows = st.tuples(ids, names, starts, notes)
tables = st.lists(rows, min_size=1)  # avoid empty tables

@given(tables)
def test_csv_round_trip(table):
    assert decode(encode(table)) == table
```

Table of contents

1 Introduction

2 Testing simple functions

3 Practice session 1

4 Testing functions with structured data

5 Practice session 2

6 Testing stateful code

7 Practice session 3

8 Conclusion

Pratice session 2

8 Use a roundtrip property of the CSV encoding example to test the "naive" encode and decode functions given in the repository:

```
@given(tables)
def test_csv_naive_round_trip(table):
    assert decode_naive(encode_naive(table)) == table
```

The tests for the naive functions will fail; can you interpret the counter-examples found?

Table of contents

1 Introduction

2 Testing simple functions

3 Practice session 1

4 Testing functions with structured data

5 Practice session 2

6 Testing stateful code

7 Practice session 3

8 Conclusion

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Testing stateful programs

We can also use Hypothesis to test code that maintain state, e.g:

- stateful objects;
- web services;
- database connections.

Rule-based State Machine Testing

- Start from a well-defined initial state
- Each transition (called a rule) can take arguments generated by strategies
- Transitions are executed both in the implementation and the model
- Compare the results of implementation against the model after each step

$$
\begin{tikzcd}
\text{implementation} & s_0 \arrow[r, "r_1"] & s_1 \arrow[r, "r_2"] \arrow[d, dash, "?"] & s_2 \arrow[r, "r_3"] \arrow[d, dash, "?"] & s_3 \arrow[r, "r_4"] \arrow[d, dash, "?"] & \cdots \\
\text{model} & m_0 \arrow[r, "r_1"] & m_1 \arrow[r, "r_2"] & m_2 \arrow[r, "r_3"] & m_3 \arrow[r, "r_4"] & \cdots
\end{tikzcd}
$$

Rule-based State Machine Testing (cont.)

- Tests fail if there is a mismatch between model and implementation
- The counter example is the rule sequence leading to the failure
- Shrinking is applied to find a minimal sequence

$$
\begin{tikzcd}
\text{implementation} & s_0 \arrow[r, "r_1"] & s_1 \arrow[r, "r_2"] \arrow[d, dash, "\checkmark"] & s_2 \arrow[r, "r_3"] \arrow[d, dash, "\checkmark"] & s_3 \arrow[d, dash, "\times"] \\
\text{model} & m_0 \arrow[r, "r_1"] & m_1 \arrow[r, "r_2"] & m_2 \arrow[r, "r_3"] & m_3
\end{tikzcd}
$$

Sequence $[r1, r2, r3]$ leads to a test failure.

First example

- A database for associating *users* and *emails*
- Implemented as a Python class and a SQLite backend
- Users are represented by an integer id
- Each user can have more than one email
- Operations:

  add(userid,email) add an email to a user
  get(userid) get all emails from a user
  remove(userid,email) remove and email from a user
  delete(userid) remove all emails from a user

First example (cont.)

```
>>> db = UserDB()
>>> db.add(1, 'foo@bar.com')
>>> db.add(2, 'baz@bong.org')
>>> db.get(1)
['foo@bar.com']
>>> db.add(1, 'foo@bing.org')
>>> db.add(1, 'foo@bar.com')
>>> db.get(1)
['foo@bar.com', 'foo@bing.org']
>>> db.remove(1, 'foo@bar.com')
>>> db.get(1)
['foo@bing.org']
>>> db.close()
```

Testing the User Database

- The model is a list of pairs (user,email)
- The state machine starts with an empty DB and empty model
- Each transition is defined as a method labeled `@rule`
  - corresponds to one of the UserDB operations
  - performs the operation in both implementation and model
  - checks that the implementation and model agree
- Hypothesis will generate random sequences of rules
- A counter-example will be a sequence of rules leading to a mismatch

(Cue demo.)

Testing the User Database (cont.)

```
AssertionError:
assert ['0@B.ac', '0@A.ac'] == ['0@A.ac', '0@B.ac']
Falsifying example:
        state = TestUserDB()
        state.add(email='0@B.ac', user=1)
        state.add(email='0@A.ac', user=1)
        state.get(user=1)
        state.teardown()
```

Fixing the testing failure

- Hypothesis found a mismatch between the implementation and model: the order of emails obtained is not always in insertion order
- We can correct this in two ways:

  strengthening the implementation fixing the order results in the SQL query

  weakening the property allow results with different orders
- More generally:
  - testing failures show that an assumption about our code does not hold
  - the error could be in the assumption, the code or both

Second example

- A simplified shopping cart for an on-line store
- Maintains quantities associated with each item
- Operations:

init(prices)	define items and prices
add(item)	add an extra item
remove(item)	remove an item (if possible)
get_subtotal()	compute current subtotal i.e. sum prices times quanties

Second example (cont.)

```
>>> c = ShoppingCart({'apples':10,'pears':20,'berries':30})
>>> c.add('apples')
>>> c.add('bananas')
ValueError: invalid item
>>> c.add('berries')
>>> c.add('apples')
>>> c.get_subtotal()
50
>>> c.remove('berries')
>>> c.get_subtotal()
20
```

Testing the shopping cart

- Unlike the previous example, we will not define an explicit model
- Instead we will just state post-conditions of each of the operations:

add	the sub-total should increase by the item's price
remove	the sub-total should decrease by the item's price

(Cue demo.)

Understanding the testing failure

```
AssertionError: assert 0 == (0 - 1)
Falsifying example:
    state = ShoppingCartMachine()
    state.start(price_dict={'A': 1, 'B': 1, 'C': 1,
                            'D': 1, 'E': 1, 'F': 1})
    state.add(item='E')
    state.remove(item='E')
    state.remove(item='E')
    state.teardown()
```

- Testing found a sequence that violated our post-conditions
- The error is caused by a bug in the implementation of the `remove` method

Table of contents

1 Introduction

2 Testing simple functions

3 Practice session 1

4 Testing functions with structured data

5 Practice session 2

6 Testing stateful code

7 Practice session 3

8 Conclusion

Practice session 3

9 Correct the error in the user email database by changing the post-condition of the `get` rule to sort the results before comparing:

```
assert sorted(answer) == sorted(expect)
```

Confirm that the tests pass with this modification.

10 Correct the error in the `remove` method of the shopping cart class so that it deletes an entry when the quantity drops to zero.

Practice session 3 (cont.)

11 Consider two extra methods for the shopping cart class in the code repository:

save exports the contents of the shopping cart to a string;

load imports the contents of the shopping cart from a string.

Modify the state machine to test these methods using the following idea:

1. define two additional fields, namely, `saved_cart` and `saved_subtotal` initially both set to `None`;
2. add a rule for save that sets the `saved_cart` and `saved_subtotal`;
3. add a rule for load with a precondition that `saved_cart` is not `None`; this should load the saved cart and verify that the subtotal equals the `saved_subtotal`.

This allows for tests that introduce some arbitrary number of cart operations between loads and saves. Check that the tests still pass.

Table of contents

1 Introduction

2 Testing simple functions

3 Practice session 1

4 Testing functions with structured data

5 Practice session 2

6 Testing stateful code

7 Practice session 3

8 Conclusion

Introduction

Testing simple functions

Practice session 1

Testing functions with structured data

Practice session 2

Testing stateful code

Practice session 3

Conclusion

Conclusion

- PBT philosophy: write properties and generate tests
- Lightweight: implemented as a library
- Flexible: a "domain-specific language" for writing strategies and properties
- Properties become "executable specifications" of the code
- Useful for finding subtle bugs in complex systems
- Useful for comunicating expectations among developers
- Conjecture: is validation of AI generated code a "killer app" for PBT?

More information

- Hypothesis documentation. `https://hypothesis.readthedocs.io/`
- D. R. MacIver *et al*. *Hypothesis: A new approach to property-based testing*, 2019.
- T. Arts *et al*. *Testing Telecoms Software with Quviq QuickCheck*, 2006
- T. Arts *et al*. *Testing AUTOSAR software with QuickCheck*, 2015
- J. Hughes *et al*. *The Mysteries of DropBox: Property-Based Testing of a Distributed Synchronization Service*, 2016
- H. Goldstein, *et al*. *Property-Based Testing in Practice*, 2024